Repository Guide

Experimentos Mestrado — Imbalanced Time Series

João P. M. Silva

CIn – UFPE

July 15, 2026

Contents

1 1. The Big Picture: What Is This Repository?

This repository is a **private, structured research laboratory** for systematically replicating 17 academic papers on imbalanced time series forecasting and regression — the core empirical work of a Master's thesis at CIn–UFPE.

1.1 1.1 What "Replication" Means Here

You are **not** re-implementing papers from scratch. The workflow is:

1. Obtain the **original authors' source code** and place it inside the repository.
2. **Run that code** in a clean, isolated virtual environment.
3. **Compare your outputs** against the results reported in the paper.
4. **Document everything**: what matched, what differed, and why.

> Your role is that of a **scientific auditor** — not a reimplementer.

This distinction matters because it defines what work is automated and what is always manual.

1.2 1.2 The Automation Boundary

The repository's design explicitly separates bookkeeping (automated) from scientific judgment (always manual).

Automated by scripts	Always done manually by you
Creating the experiment folder structure	Cloning or copying the original source code
Replacing placeholder text in template files	Downloading datasets
Rebuilding the progress dashboard in `README.md`	Running the experiments
Syncing paper metadata from the Excel spreadsheet	Interpreting results and filling the comparison table
Generating this guide as a PDF	Marking an experiment as Done

The most important rule: **you control all data and code insertion**. Scripts only handle administrative bookkeeping.

1.3 1.3 Repository Goals

By the end of the thesis, this repository should contain:

- **17 fully replicated experiments**, each with a complete scientific report.
- **Documented deviations** from original paper results, with explanations.
- A **paper registry** of all 43 papers in the reading list, with replication status.
- **Shared utilities** for metrics and visualization that are reusable across experiments.

1.4 1.4 Reading This Guide

This guide is organized in the order you will need the information:

Chapter	When you need it
2 — Directory Structure	First session: orienting yourself
3 — Experiment Lifecycle	Every experiment: knowing what stage you are in
4 — Core Workflow	Every session: the exact step-by-step protocol
5 — Automation Scripts	When running or understanding the scripts
6 — File Reference	When filling in any experiment file
7 — Environments	When setting up a new Python environment
8 — Git Workflow	After each work session
9 — Adding Experiments	When adding experiment #018 or beyond
10 — Rules	Quick reference cheatsheet
11 — Troubleshooting	When something breaks

2 2. Directory Structure

This chapter is the canonical reference for the repository layout. Every top-level folder has a single, well-defined purpose. Understanding this structure is a prerequisite for all other work.

2.1 2.1 Full Layout

```
experimentos_mestrado_imbalnced_ts/
│
├── README.md                    <- Master dashboard (auto-updated, never edit by
↪  hand)
├── CONTRIBUTING.md              <- The rules: how to add / maintain experiments
├── .gitignore                   <- What Git ignores: venvs, datasets, build
↪  artifacts
│
├── experiments/                 <- One subfolder per paper replication
│   │
│   ├── _template/               <- Blueprint. NEVER edit directly.
│   │   ├── README.md            <- Structured experiment report (fill this in)
│   │   ├── REPLICATION_LOG.md   <- Daily work diary (prepend new entries)
│   │   ├── requirements.txt     <- Python dependencies for this experiment
│   │   ├── config/
│   │   │   └── default.yaml     <- Key parameters: seeds, paths, hyperparams
│   │   ├── src/
│   │   │   ├── original/        <- AUTHORS' code goes here (manual, never
↪  scripted)
│   │   │   └── adapted/         <- YOUR changes and wrappers go here
│   │   ├── notebooks/           <- Exploratory Jupyter notebooks
│   │   └── results/
│   │       ├── tables/          <- CSV / XLSX output tables
│   │       └── figures/         <- Generated plots and visualizations
│   │
│   ├── 001_uba_utility_regression/
│   ├── 002_ts_resampling_strategies/
│   ├── ... (17 experiment folders in total)
│   └── 017_imbalanced_regression_pipeline/
│
├── scripts/                     <- Automation tools (bookkeeping only)
│   ├── new_experiment.sh        <- Scaffolds a new experiment from _template
│   ├── update_readme.py         <- Rebuilds the progress dashboard in README.md
│   └── sync_from_excel.py       <- Syncs paper metadata from Google Drive Excel
│
├── shared/                      <- Code reused across multiple experiments
│   ├── datasets/
│   │   └── README.md            <- Registry of shared datasets and download
↪  instructions
│   ├── metrics/
```

```
│   │   ├── regression.py         <- RMSE, MAE, SMAPE — fallback only; prefer
↪  paper's own
│   │   └── classification.py     <- F1, Precision, Recall — fallback only
│   ├── visualization/
│   │   └── plot_utils.py         <- Publication-quality matplotlib helpers
│   └── utils/
│       └── data_loading.py       <- Path helpers for navigating the repo structure
│
├── docs/                         <- Reference documents and this guide
│   ├── guide/                    <- Source for this PDF
│   ├── paper_registry.md         <- Auto-generated registry of all 43 papers
│   └── methodology.md            <- Scientific replication methodology
│
└── .github/
    └── ISSUE_TEMPLATE/
        └── new-experiment.md     <- GitHub issue template for proposing new
        ↪  experiments
```

2.2 2.2 Folder Reference

2.2.1 `experiments/` — The Core of the Repository

This is where all research lives. Each subfolder corresponds to one paper replication and follows a strict naming convention:

```
NNN_descriptive_name/
```

Where `NNN` is a zero-padded three-digit number that encodes the **priority order** of the experiment. Lower numbers are done first.

Golden rules for `experiments/`:

- Never add a folder by hand. Always use `new\{}_experiment.sh`.
- Never rename an existing folder. The number is the experiment's permanent identifier.
- The folder structure inside each experiment must match `\{}_template/` exactly.

2.2.2 `experiments/_template/` — The Blueprint

This folder defines the schema for every experiment in the repository. The automation script `new\{}_experiment.sh` copies it in its entirety when creating a new experiment.

> **Critical rule:** Never edit `\{}_template/` to fix a single experiment. If you edit the template, you are changing the contract for all *future* experiments. Edit the specific experiment folder instead.

Contents of each experiment folder:

File / Folder	Purpose	Who fills it
`README.md`	Structured scientific report: metadata, results, status	You
`REPLICATION\{}_LOG.md`	Chronological work diary, newest entry on top	You
`requirements.txt`	Exact Python package versions (`pip freeze`)	You
`config/default.yaml`	Experiment parameters: seeds, paths, hyperparams	You
`src/original/`	Authors' original source code, cloned or copied	You (manual)
`src/adapted/`	Your modifications, wrappers, and analysis code	You
`notebooks/`	Jupyter notebooks for exploration	You
`results/tables/`	Output CSVs and comparison tables	Experiments
`results/figures/`	Generated plots	Experiments

2.2.3 `scripts/` — Automation Tools

These scripts handle **bookkeeping only**. They never touch your data or experiment results.

Script	When to run	What it does
`new\{}_experiment.sh`	Adding an experiment after #017	Copies `\{}_template/`, renames placeholders, sets date
`update\{}_readme.py`	After changing any experiment's status	Rebuilds the status table in the root `README.md`
`sync\{}_from\{}_excel.py`	After updating the Google Drive paper spreadsheet	Regenerates `docs/paper\{}_registry.md`

2.2.4 `shared/` — Cross-Experiment Code

Code that is genuinely reused across multiple experiments lives here instead of being duplicated inside each experiment folder.

Important usage rule: The `shared/metrics/` implementations are **fallback implementations** only. If the paper provides its own metric code, use that. Shared metrics exist to avoid duplication when no reference implementation is available.

2.2.5 `docs/` — Documentation

The `docs/` folder contains reference material about the project as a whole, not about any individual experiment.

File	Description
`guide/`	Source Markdown files for this PDF guide
`paper\{}_registry.md`	Auto-generated list of all 43 papers from the Excel spreadsheet
`methodology.md`	The scientific replication methodology (what counts as a valid replication)

2.3 2.3 The Two Critical Boundaries

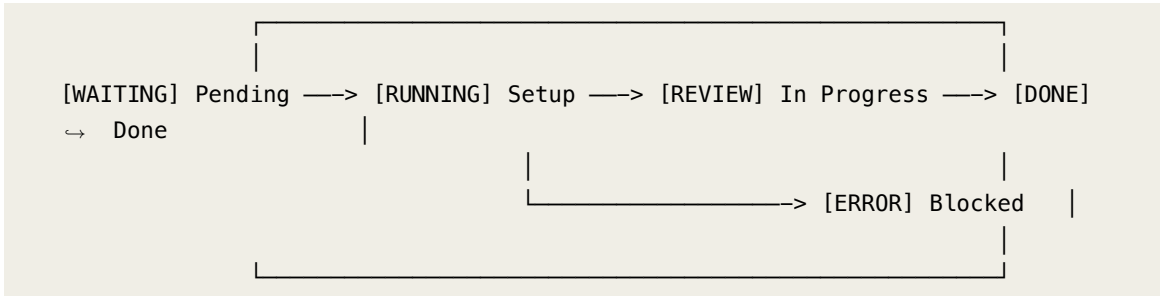
These two rules are the most important structural decisions in the repository. Violating them breaks traceability.

Boundary 1 — `\{}_template/` is immutable per experiment. The script creates your experiment folder from the template. After that, only *you* edit the experiment folder. The template is never touched again for that experiment.

Boundary 2 — `src/original/` is always filled manually. No script will ever write to `src/original/`. You clone or copy the authors' code there. This makes the provenance of all code 100% explicit and auditable.

2.4 3. The Experiment Lifecycle: From Pending to Done

Every experiment goes through exactly these stages. The status badge in each experiment's `README.md` must always reflect the current stage.



2.4.1 Stage Definitions

Badge	Stage	What it means	When to set it
[WAITING]	**Pending**	Folder exists, no work started	Default state for all 17 experiments
[RUNNING]	**Setup**	Environment is working	After: code is in `src/original/`, dependencies are installed, and the code runs without import errors
[REVIEW]	**In Progress**	Actively producing results	After: first successful end-to-end run, target results identified
[DONE]	**Done**	Replication complete	After: key results match the paper (within tolerance), README comparison table is filled, REPLICATION_LOG is complete
[ERROR]	**Blocked**	Cannot proceed	When: broken deps, missing data, or unresolvable errors — **always document the blocker**

How to change the status: Open the experiment's `README.md` and change the line that says:

```
## Replication Status: [WAITING] Pending
```

to the appropriate badge. Then run `python3 scripts/update\{}_readme.py` to propagate the change to the main dashboard.

2.5 4. The Core Workflow: Step-by-Step Instructions

This is the exact sequence you must follow every time you work on a replication.

2.5.1 Phase 0: Starting a New Session

Before doing anything else at the start of each work session:

```
# 1. Navigate to the repository
cd /Users/joaopms/Documents/Projeto_Mestrado

# 2. Check what's currently happening
git status
git log --oneline -5
```

This tells you exactly where you left off.

2.5.2 Phase 1: Picking an Experiment

Open `README.md` in your browser or editor. The Experiment Index table shows all 17 experiments and their current status. Work in **priority order**:

- **P0 first** (001–005): Core thesis papers — resampling strategies for imbalanced regression
- **P1 second** (006–011): Methodology papers — performance estimation, model selection
- **P2 after** (012–015): Extended scope
- **P3 last** (016–017): Reference implementations

Pick the experiment with the **lowest number** that is not yet `[DONE] Done`.

2.5.3 Phase 2: Setting Up the Experiment Environment

This is the most important and most manual phase. Do it in order.

Step 2.1 — Read the paper and the ARA— Before touching any code, read: 1. The experiment's `README.md` — understand the target results you need to reproduce 2. The corresponding ARA in your Obsidian Vault (if it exists) — it contains distilled information about the paper's methodology and code

Step 2.2 — Copy the original code into `src/original/`— This is **always done manually**. You have two options:

Option A: Clone the repository

```
cd experiments/NNN_folder_name/src/
git clone https://github.com/author/repo original
```

> [WARN] Do NOT use `git clone` inside the main repo without careful `.gitignore` handling. It's safer to clone elsewhere, then copy the folder.

Option B: Copy downloaded files

```
# After downloading the zip and extracting:
cp -r ~/Downloads/author-repo/ experiments/NNN_folder_name/src/original/
```

Step 2.3 — Create an isolated Python environment— **CRITICAL: Each experiment must have its own isolated environment.** Never install packages globally.

```
cd experiments/NNN_folder_name/

# Create the virtual environment
python3 -m venv .venv

# Activate it
source .venv/bin/activate

# Confirm you're in the right environment
which python3
# Should show: .../experiments/NNN_folder_name/.venv/bin/python3
```

Step 2.4 — Install dependencies—

```
# Option A: If the paper has a requirements.txt
pip install -r src/original/requirements.txt

# Option B: If the paper has a setup.py or pyproject.toml
pip install -e src/original/

# Option C: Install manually (document every package you install)
pip install numpy pandas scikit-learn

# Save what you installed
pip freeze > requirements.txt
```

Step 2.5 — Test that everything imports without error—

```
# Quick smoke test — confirm no broken imports
python3 -c "import sys; sys.path.insert(0, 'src/original'); print('imports OK')"
```

Step 2.6 — Change status to [RUNNING] Setup and commit— Once the environment works: 1. Open `experiments/NNN\{}_folder\{}_name/README.md` 2. Change `\{}#\{}# Replication Status: [WAITING] Pending` to `\{}#\{}# Replication Status: [RUNNING] Setup` 3. Run the dashboard update: `bash python3 scripts/update\{}_readme.py` 4. Commit: `bash git add experiments/NNN\{}_folder\{}_name/ README.md git commit -m "[EXP-NNN] setup: clone original code and create venv" git push origin main`

2.5.4 Phase 3: Running the Replication

Step 3.1 — Identify the exact target results— Read the paper carefully. Decide which specific tables and figures you will reproduce. Add them to the experiment's `README.md` under the `Objective` section:

```
## Objective

**Target results to replicate**:
- [ ] Table 3: RMSE comparison across resampling strategies on 5 datasets
- [ ] Figure 2: Performance vs. imbalance ratio curve
```

Step 3.2 — Run the original code with the original parameters— Always start with the **exact original parameters** the authors used. Do not change anything on the first run.

```
# Activate the environment first
source experiments/NNN_folder_name/.venv/bin/activate

# Navigate to the experiment directory
cd experiments/NNN_folder_name/

# Run the original code (the exact command varies by paper)
python3 src/original/main.py
# or: Rscript src/original/run_experiments.R
```

Step 3.3 — Update the REPLICATION_LOG immediately— Every time you do something, write it down in `REPLICATION\{}_LOG.md`. This is your scientific diary. Entries go at the **top** of the file (most recent first).

```
## [2026-07-15] First successful run attempt

**Time spent**: 2h 30m

### What was done
- Activated .venv and ran src/original/main.py
- Got a FileNotFoundError for the AEMO dataset

### What worked
- All imports resolved correctly
- Configuration loading worked

### Issues encountered
- The paper's data is not publicly available. Authors reference "private AEMO
↪  dataset".
- Contacted authors via email.

### Next steps
- Try with the public NASDAQ dataset as a proxy
- Wait for author response
```

Step 3.4 — Change status to [REVIEW] In Progress— Once you have your first successful run, update the status badge in the experiment's `README.md` from `[RUNNING]` to `[REVIEW]`.

2.5.5 Phase 4: Recording Results

Step 4.1 — Save results to the right location— All outputs **must** go inside the experiment's `results/` directory: - Tables -> `results/tables/` as `.csv` files - Figures -> `results/figures/` as `.png` (and optionally `.pdf`) files

Name files descriptively:

```
results/
├── tables/
│   ├── table3_rmse_comparison.csv
│   └── table4_mae_by_dataset.csv
└── figures/
    ├── figure2_performance_vs_imbalance.png
    └── figure5_training_time.png
```

Step 4.2 — Fill the comparison table in README.md— This is the most important scientific output. For each metric you reproduce:

```
## Key Results

| Metric | Paper Reports | Our Replication | Δ (absolute) | Notes |
|--------|---------------|-----------------|--------------|-------|
| RMSE (AEMO dataset) | 0.342 | 0.349 | +0.007 | Different random seed |
| SERA (AEMO dataset) | 0.891 | 0.887 | −0.004 | Within acceptable range |
```

Step 4.3 — Document deviations honestly— In the `Observations \{}& Deviations` section:

```
## Observations & Deviations

- **Library version**: Original used scikit-learn 0.24.2; we used 1.3.0.
  The API for `RandomForestRegressor` changed in v1.0; adjusted accordingly.
- **Dataset**: Original code references a private AEMO energy dataset.
  We used the publicly available GEFCom2014 as a proxy. Results are directionally
  ↪  consistent.
- **Seed**: Original paper does not specify a seed. We used seed=42 for
↪  reproducibility.
```

2.5.6 Phase 5: Marking Complete and Committing

Step 5.1 — Mark checkboxes done in README— Check off every target result you successfully reproduced:

```
**Target results to replicate**:
- [x] Table 3: RMSE comparison across resampling strategies on 5 datasets
- [x] Figure 2: Performance vs. imbalance ratio curve
```

Step 5.2 — Change status to [DONE] Done— Change `\{}#\{}# Replication Status: [REVIEW] In Progress` to `\{}#\{}# Replication Status: [DONE] Done`.

Step 5.3 — Run the update script and commit—

```
# Update the main dashboard
python3 scripts/update_readme.py

# Stage all changes
git add experiments/NNN_folder_name/ README.md

# Commit with a descriptive message
git commit -m "[EXP-NNN] results: reproduce Table 3 and Figure 2"

# Push
git push origin main
```

2.6 5. The Automation Scripts: What They Do and When to Use Them

2.6.1 `new_experiment.sh` — Scaffolding New Experiments

When to use it: Only when adding an experiment **beyond the 17 already scaffolded** (i.e., experiment 018 onwards).

What it does: 1. Copies the entire `experiments/\{}_template/` directory to a new folder. 2. Replaces all placeholder text (`[PAPER\{}_TITLE]`, `[CODE\{}_URL]`, etc.) in the new folder's files. 3. Sets today's date in the `REPLICATION\{}_LOG.md`.

How to use it:

```
./scripts/new_experiment.sh NNN "folder_name" "Paper Title" "https://code-url"
```

Real example:

```
./scripts/new_experiment.sh 018 "smogn_regression" "SMOGN: a Pre-processing Approach
↪  for Imbalanced Regression" "https://github.com/nicktaonline/smogn"
```

After running it, you will see:

```
[DIR] Creating experiment: 018_smogn_regression
[OK] Experiment scaffolded at:
↪  /Users/joaopms/Documents/Projeto_Mestrado/experiments/018_smogn_regression

Next steps:
  1. Edit README.md — fill in metadata fields
  2. Clone original code into src/original/
  3. Set up environment and update requirements.txt
  4. Run: python3 scripts/update_readme.py
  5. Commit: git add experiments/018_smogn_regression/ ...
```

> [WARN] **Do NOT run this script** for experiments 001–017. They are already scaffolded.

2.6.2 `update_readme.py` — Refreshing the Dashboard

When to use it: Every time you change the status badge in any experiment's `README.md`.

What it does: 1. Scans every folder under `experiments/NNN\{}_*/` for a `README.md`. 2. Reads the `\{}#\{}# Replication Status: [badge]` line from each one. 3. Updates the progress summary table (Done / In Progress / Setup / Pending / Blocked counts). 4. Updates today's date in the "Last updated" field.

How to use it:

```
# Dry run — shows what WOULD change, but writes nothing
python3 scripts/update_readme.py --check

# Live run — actually updates README.md
python3 scripts/update_readme.py
```

When to run it: - Whenever you change an experiment's status badge - Before every `git commit` that involves progress changes

> **Important:** This script only updates the progress tables. It does NOT change the Experiment Index table. If you add a brand new experiment and want it in the index, you must manually add a row there.

2.6.3 `sync_from_excel.py` — Syncing Paper Metadata

When to use it: When you update the `Fichamento\{}_Artigos.xlsx` file in Google Drive and want the changes reflected in `docs/paper\{}_registry.md`.

What it does: 1. Opens `Fichamento\{}_Artigos.xlsx` from your Google Drive (path is hardcoded in the script). 2. Reads all rows from the first sheet. 3. Groups papers into "with code" and "without code" based on the `Disponibilidade de Códigos?` column. 4. Writes a formatted markdown table to `docs/paper\{}_registry.md`.

How to use it:

```
python3 scripts/sync_from_excel.py
```

Expected output:

```
[OK] Paper registry written to: .../docs/paper_registry.md
   17 papers with code, 26 without code
```

[WARN] **Requirement:** `openpyxl` must be installed (`pip3 install openpyxl`).
[WARN] **Requirement:** Google Drive must be mounted and the file accessible at its hardcoded path.

2.7 6. File-by-File Reference: What to Write Where

2.7.1 `experiments/NNN_folder_name/README.md`

This is the most important file in each experiment. Think of it as the **scientific report** for that replication.

Section	What to write
Paper Metadata	Fill in Authors, Year, Venue, DOI, Code URL, language, ARA link, Excel row
Replication Status	Keep this badge accurate — it drives the master dashboard
Objective	List exactly which tables/figures you plan to reproduce. Check them off when done.
Environment Setup	Write the exact commands to set up the environment from scratch. Assume a reader starts with nothing.
How to Run	Write the exact commands to run the experiment.
Key Results	Fill in the comparison table. This is the scientific output.
Observations & Deviations	Be honest about differences from the paper.

2.7.2 `experiments/NNN_folder_name/REPLICATION_LOG.md`

This is your **daily diary**. Write in it every time you work on the experiment. New entries go at the **top**.

Format:

```
## [YYYY-MM-DD] What You Did

**Time spent**: Xh Xm

### What was done
- ...

### What worked
- ...

### Issues encountered
- ...

### Next steps
- ...
```

2.7.3 `experiments/NNN_folder_name/requirements.txt`

Contains the **Python packages** needed to run the experiment. Update it after setting up the environment:

```
pip freeze > requirements.txt
```

2.7.4 `experiments/NNN_folder_name/config/default.yaml`

The **experiment configuration** file. Use it to track the key parameters of your replication run: - Random seeds - Dataset paths - Model hyperparameters - Evaluation settings

This gives you a single place to see what parameters produced your results.

2.7.5 `shared/metrics/regression.py` and `shared/metrics/classification.py`

Only use these if the original paper doesn't provide its own metric implementations. These are convenience implementations. The paper's own metrics always take priority.

2.8 7. Environment Management: Python, R, and Conda

2.8.1 Python Experiments (most common)

```
cd experiments/NNN_folder_name/

# Create the isolated environment
python3 -m venv .venv

# Activate (macOS/Linux)
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# When done working, deactivate
deactivate
```

> [IDEA] The `.venv/` directory is listed in `.gitignore` — it will NOT be committed. That's correct. The `requirements.txt` is what gets committed.

2.8.2 R Experiments

For papers using R (e.g., UBA, IRon):

```
cd experiments/NNN_folder_name/

# Open R and install the needed packages
Rscript -e "install.packages(c('uba', 'performanceEstimation', 'randomForest'))"

# Create an install script for reproducibility
cat > install_packages.R << 'EOF'
install.packages(c("uba", "performanceEstimation"))
EOF
```

Document the R version in the README:

```
R --version | head -1
```

2.8.3 Conda Experiments

For papers that require specific CUDA or complex binary dependencies:

```
conda env create -f src/original/environment.yml
conda activate experiment-name
```

2.9 8. Git Workflow: How and When to Commit

2.9.1 The Commit Message Convention

Format: `[EXP-NNN] action: description`

Action	When	Example
`setup`	First scaffolding, cloning code	`[EXP-001] setup: clone UBA code and create venv`
`run`	Running experiments, generating results	`[EXP-001] run: reproduce Table 3 with original params`
`fix`	Fixing bugs or dependency issues	`[EXP-003] fix: adjust data path for local AEMO dataset`
`results`	Adding result tables or figures	`[EXP-001] results: add comparison table for RMSE metrics`
`docs`	Updating README or REPLICATION_LOG	`[EXP-001] docs: complete REPLICATION\{}_LOG for session`
`refactor`	Restructuring code	`[EXP-001] refactor: move helper functions to adapted/`

For repo-wide changes:

```
[REPO] docs: update README progress summary
[REPO] feat: add shared SERA metric implementation
```

2.9.2 When to Commit

Commit after each of these milestones: 1. After scaffolding environment (status -> [RUNNING]) 2. After first successful run (status -> [REVIEW]) 3. After generating each set of results 4. After completing the experiment (status -> [DONE]) 5. After any significant REPLICATION_LOG update

2.9.3 The Standard Commit Sequence

```
# 1. Check what changed
git status

# 2. Stage only what's relevant
git add experiments/NNN_folder_name/
git add README.md  # If you ran update_readme.py

# 3. Verify what's staged
git diff --staged --stat

# 4. Commit
git commit -m "[EXP-NNN] action: short description"
```

```
# 5. Push to GitHub
git push origin main
```

[WARN] **Never commit:** `*.csv` data files, `*.h5` model files, `.venv/` directories, or `\{}_\{}_pycache\{}_\{}_/`. These are all in `.gitignore`.

2.10 9. Adding a Brand New Experiment (Beyond the 17)

If you find a new paper you want to replicate that is not in the current index:

Step 1 — Run the scaffolding script—

```
./scripts/new_experiment.sh 018 "new_paper_name" "Full Paper Title"
↪  "https://github.com/author/repo"
```

Step 2 — Add a row to the Experiment Index in README.md— Open `README.md` and add a new row to the `\{}#\{}# [INDEX] Experiment Index` table:

```
| 018 | Full Paper Title | 2025 | Author Names | [WAITING] Pending |
↪  [GitHub](https://github.com/...) | [->](experiments/018_new_paper_name/) |
```

Step 3 — Update `docs/paper_registry.md`— If the paper is in the Excel file, run:

```
python3 scripts/sync_from_excel.py
```

If it's not, add it to the Excel file first, then run the sync script.

Step 4 — Commit—

```
git add experiments/018_new_paper_name/ README.md docs/paper_registry.md
git commit -m "[REPO] feat: scaffold experiment 018 for Full Paper Title"
git push origin main
```

2.11 10. Rules and Conventions You Must Never Break

These rules exist to ensure the repository stays usable 6 months from now.

[FAIL] Never	[OK] Always
Edit `experiments/\{}_template/` for a one-off reason	Edit the specific experiment folder instead
Commit large data files (`.csv`, `.h5`, etc.)	Document the download source in the README
Commit the `.venv/` directory	Use `requirements.txt` for reproducibility
Change an experiment's status without running the update script	Run `python3 scripts/update\{}_readme.py` after every status change
Write results outside `results/tables/` and `results/figures/`	Save tables as `.csv`, figures as `.png`
Skip the REPLICATION_LOG when things go wrong	Document failures in detail — they are scientifically valuable
Use generic commit messages like "update"	Use the `[EXP-NNN] action:` format
Install packages in the global Python environment	Always activate the experiment's `.venv` first

2.12 11. Troubleshooting Common Problems

2.12.1 "I ran `update_readme.py` but the status didn't change"

Cause: The status line in the experiment's `README.md` doesn't match the expected format.

Check: The status line must look **exactly** like:

```
## Replication Status: [WAITING] Pending
```

Any extra spaces, different emoji, or different heading level will cause the script to miss it.

2.12.2 "I get a permission denied error running `new_experiment.sh`"

Solution:

```
chmod +x scripts/new_experiment.sh
```

2.12.3 "The `sync_from_excel.py` script says the Excel file is not found"

Cause: Google Drive is not mounted or the file path has changed.

Check:

```
ls "/Users/joaopms/Library/CloudStorage/GoogleDrive-jpms5@cin.ufpe.br/My
↪  Drive/Papers Mestrado - João/Fichamento_Artigos.xlsx"
```

If the file doesn't exist there, make sure Google Drive is open and synced.

2.12.4 "I accidentally committed a large data file"

Solution:

```
# Remove the file from Git tracking but keep it locally
git rm --cached path/to/large/file.csv

# Add it to .gitignore
echo "experiments/NNN_folder_name/data/file.csv" >> .gitignore

# Commit the removal
git add .gitignore
git commit -m "[REPO] fix: untrack large data file from EXP-NNN"
git push origin main
```

2.12.5 "I'm not sure which experiment to work on next"

Open `README.md`. The progress summary table shows the count in each status. Work on the lowest-numbered experiment that is `[WAITING] Pending` or `[ERROR] Blocked` — in that priority order.

2.12.6 "The original code doesn't run at all — different Python version, broken deps"

This is common! Recommended approach: 1. Try installing the **exact versions** specified in the original `requirements.txt` (if it exists). 2. If that fails, try a `conda` environment with the paper's stated Python version. 3. Check GitHub Issues on the original repository — others may have solved it. 4. Document the problem in `REPLICATION\{}_LOG.md` and set status to `[ERROR] Blocked`. 5. Try the next experiment and come back later.

Last updated: 2026-07-10 Repository: https://github.com/jpmsilva1/experimentos_mestrado_imbalnced_ts

A Replication Methodology

This document describes the systematic methodology used for replicating experiments from the literature.

A.1 Definition of "Replication"

In this repository, **replication** means:

1. **Obtaining** the original source code from the paper's public repository (**done manually by the user** — copied into `src/original/`)
2. **Setting up** the environment (dependencies, data, configuration)
3. **Running** the original code with the original parameters
4. **Comparing** our output with the results reported in the paper
5. **Documenting** any deviations, issues, or insights

We are **NOT** reimplementing from scratch. We are verifying that the original code produces the claimed results.

> **Note**: All code, dataset, and file insertion is a **manual process** performed by the researcher. The repository scaffolding provides the structure and conventions; the user populates each experiment folder at their own pace.

A.2 Replication Criteria

An experiment is considered **successfully replicated** when:

- The original code runs without errors in our environment
- Key metrics (RMSE, MAE, F1, SERA, etc.) match the paper within a **reasonable tolerance** (±5% for stochastic methods, exact match for deterministic)
- Any deviations are explained (different random seeds, library versions, data splits)

A.3 Standard Workflow per Experiment

1. **Read the paper** — understand the claims, methodology, and target results
2. **Read the ARA** (if exists) — review compiled knowledge in the Obsidian Vault
3. **Insert the code** — manually copy/clone the original code into `src/original/` (user-owned step)
4. **Insert datasets** — manually place required data files locally (user-owned step)
5. **Set up environment** — create isolated `venv` or `conda` env
6. **Run original code** — with original parameters, document the process
7. **Record results** — save outputs to `results/`, fill comparison table
8. **Document** — complete README and REPLICATION_LOG
9. **Update dashboard** — run `scripts/update\{}_readme.py`

A.4 Evaluation Metrics Reference

Metric	Domain	Description
RMSE	Regression	Root Mean Squared Error
MAE	Regression	Mean Absolute Error
SERA	Imbalanced Regression	Squared Error-Relevance Area (from IRon)
F1	Classification	Harmonic mean of precision and recall
AUC-ROC	Classification	Area under ROC curve
TSS	Classification	True Skill Statistic (used in solar flare prediction)

A.5 Tools & Infrastructure

- **Python**: Primary language (most experiments)
- **R**: Secondary language (UBA, IRon, some Torgo group papers)
- **Git**: Version control for all code and results
- **Obsidian Vault**: Cross-referenced knowledge base (ARAs)
- **W&B / MLflow**: Optional experiment tracking for complex runs